

CS 498 — Stage 2 Report

Adam El Bahey (elbahey2) — Data Analyst

Team: Srikar Pisupati, Anirudh Venkatraman, Adam El Bahey

Project Overview

For our cloud data management project, our team built an analytical prototype on top of the Twitter Eurovision 2018 dataset — roughly 800,000 raw tweets distributed across two NDJSON files totaling 3.7 GB. The goal of the project was to choose a cloud-friendly storage system, design a schema that fit the dataset, ingest the full corpus, and answer a fixed set of six analytical queries about user behavior, geographic activity, hashtag popularity, and reply-graph structure. Because tweets arrive as nested JSON objects with optional fields, irregular shapes, and embedded arrays, the central design question was which storage paradigm would let us answer those queries cheaply without warping the data on the way in.

After comparing **MongoDB**, **Apache Cassandra**, and **Redis** during Stage 1, we converged on MongoDB. Cassandra's wide-column model is optimized for predictable, primary-key-driven access patterns, but our queries were inherently exploratory: group-bys on country, \$unwind operations on hashtag arrays, and graph traversals on reply edges. Cassandra discourages secondary indexes at scale and offers no native joins or aggregations comparable to MongoDB's pipeline. Redis was attractive for raw read latency, but its key-value model is a poor fit for nested document analytics, and its 512 MB-class instances would have required us to flatten and shard manually. MongoDB's BSON document model maps almost directly onto raw tweet JSON, its multikey indexes natively support hashtag arrays, and its aggregation framework provides SQL-like analytical primitives (\$match, \$group, \$unwind, \$sort, \$limit) over those documents. For a dataset that is fundamentally semi-structured and read-heavy, MongoDB was the system that demanded the least friction between the data and the questions we wanted to ask.

Our final schema kept one document per tweet but flattened the fields the six queries actually touch: tweet id, created_at, user identity and verification, reply linkage, hashtags, and place country. We rejected two alternatives during design: storing each raw tweet untouched (which would force every query to navigate a deep, inconsistent shape) and splitting into normalized collections for users, tweets, hashtags, and replies (which would have introduced join-like \$lookup stages MongoDB is not optimized for). The middle-ground design — one document per tweet, with a precomputed tweet_type and lowercased entities.hashtags — gave us indexable, deterministic access paths for every query.

The ingestion pipeline was a Python script using pymongo. It streams each NDJSON file line-by-line, skips stream-control records (the dataset contains roughly 22,000 {"limit": ...} and similar records that are not tweets), normalizes hashtags to lowercase so #Eurovision and #eurovision collapse together, and bulk-writes documents in batches of 5,000. After load, two indexes support our two demonstrated queries: a single-field index on place.country and a multikey index on entities.hashtags. The final collection contains **795,473 tweets** in 39 MB on disk. Loading runs in 173 seconds against a local MongoDB instance and 23 minutes against MongoDB Atlas on Google Cloud — the latter being network-bound

rather than compute-bound. Once indexed, **Q2 (country with most tweets)** returns in 0.1 seconds with **United Kingdom** at the top with 6,041 geo-tagged tweets, and **Q4 (top 100 hashtags)** completes in 6.5 seconds locally (3.1 seconds on Atlas) and identifies #eurovision as the dominant tag with 591,213 occurrences, followed by #esc2018 and #allaboard.

My Role as Data Analyst

My responsibility on this team was the data-analyst role: turning the team's storage and schema decisions into actual analytical answers, validating that those answers were defensible, and feeding query patterns back into the schema and indexing decisions. Over the past few weeks I worked through every one of the six required queries on paper before any of them were implemented, mapping each to its required access path and confirming that our schema exposed the right fields. That exercise is what surfaced two design changes from our initial sketch: pushing tweet_type to be precomputed at ingest (so the verified-user breakdown query becomes a simple grouped aggregation rather than a string-comparison-heavy filter) and lowercasing hashtags during ETL (so the top-100 hashtag query is not polluted by case variants of the same tag).

For the two queries we are demonstrating, I authored the aggregation pipelines, validated the results against my own intuition about the dataset, and ran them in two environments to verify portability. The local MongoDB instance and the Atlas GCP-hosted cluster returned identical document counts and identical top-of-list results — a basic but important check that our pipeline is reproducible and not dependent on a single machine. I also performed sanity passes on the output: the United Kingdom result is consistent with the Eurovision audience profile, and the dominance of #eurovision followed by #esc2018 reflects the official versus shorthand naming convention used by Eurovision broadcasters in 2018. These cross-checks are the kind of work that distinguishes a query that ran successfully from a result that is actually trustworthy.

Beyond the demo queries, I contributed to the team's index strategy by listing every query's required filter and group-by fields, identifying which would benefit from compound indexes versus single-field indexes, and arguing for the multikey index on entities.hashtags specifically because the top-100 query relies on \$unwind and would otherwise require a collection scan. Finally, I prepared the analytical results for downstream presentation: producing the JSON exports of the top-100 hashtags and the full country ranking that the team uses in the Stage 2 deliverable, and writing the supporting commentary that explains why the numbers look the way they do.